PANAMA BLOT ANALYZER 0.2.0

# Documentação técnica do código

## Arquitetura, dados, algoritmos e manutenção

Descrição detalhada da implementação local do Panama Blot Analyzer, organizada para quem precisa compreender, auditar ou modificar o programa. A leitura acompanha o caminho de uma imagem até as medidas, a PCA e os arquivos persistidos.

**Relação com o Nirvana**
A documentação combina a explicação de método presente na introdução original com um guia das funções da reimplementação Python/JavaScript. As correspondências históricas estão separadas das mudanças e dos recursos novos.

**Escopo verificado**
main.py; panama/analysis.py; panama/model.py; panama/server.py; panama/web/app.js; arquivos de interface; testes; scripts de construção; documentação de métodos e registros do Mm01a.

A base local se identifica como **0.2.0**. O formato do projeto permanece na versão **1**. O repositório local não possui commit HEAD; por isso, o apêndice registra hashes dos arquivos principais em vez de um identificador de commit.

Edição de 4 de novembro de 2026
Complementa o Manual de uso em português. As instruções de compilação descrevem o ambiente previsto pelo projeto, sem introduzir alterações na aplicação.

DOCUMENTAÇÃO TÉCNICA

# Conteúdo e roteiro de leitura


| Parte | Página |
|---|---|
| Arquitetura e ciclo de uma operação | 3 |
| Modelo de dados e estado derivado | 4 |
| Leitura de imagens e perfis | 5 |
| Extração, detecção de faixas e recorte | 6 |
| Detecção de marcos e filtro FFT | 7 |
| Correspondência e transformação de coordenadas | 8 |
| Linha de base, artefatos e integração | 9 |
| Construção da matriz e PCA | 10 |
| Interpretação computacional e exemplo numérico | 11 |
| Comandos, desfazer e invalidação | 12 |
| Formato .panama e exportação | 13 |
| Servidor HTTP e ponte desktop | 14 |
| Interface e mapa de operações | 15 |
| Execução, dependências e testes | 16 |
| Correspondência histórica e limites | 17 |
| Fontes, hashes e manutenção | 18 |
| Análises e exportação Excel | 19 |
| Estatística, pareamento e testes | 20 |


Leia main.py, Engine.command() e analyze_project() para acompanhar inicialização, edição e cálculo. As novas análises estão em multivariate.py; a exportação Excel, em workbook.py.

O estado Python é a fonte dos cálculos. W e P representam blot e proteína.

# 1. Arquitetura e fluxo

| Arquivo / componente | Responsabilidade |
|---|---|
| main.py | Argumentos de execução, janela pywebview, fallback de navegador e diálogos nativos. |
| panama/server.py | Servidor HTTP local, autenticação da sessão, serialização JSON e exclusão mútua. |
| panama/model.py | Modelo do experimento, Engine, validação, comandos, histórico, ZIP do projeto e exportação. |
| panama/analysis.py | Importação numérica, extração de imagem, alinhamento, fundo, integração, PCA e clustering. |
| panama/web/ | index.html: estrutura; style.css: apresentação; app.js: interação, estado visual e gráficos SVG. |
| tests/ e tools/ | Testes automatizados; extração da referência histórica; ensaio e documentação do Mm01a. |

## Caminho de uma edição

```
Clique ou edição em app.js
  -> api(action, params)
  -> POST /api/command
  -> Engine.command() -> _apply()
  -> validate_project() -> analyze_project()
  -> Engine.public() -> JSON
  -> atualização de state -> render()
```

O navegador mantém **state** com a representação pública do motor e **ui** com seleção de gel/amostra, aba, ferramenta e zoom. A edição é enviada ao Python. Se a aplicação do comando ou o recálculo falhar, o motor restaura o estado anterior e devolve um erro.

O servidor usa ThreadingHTTPServer, mas um RLock protege acesso ao motor nas rotas e nos diálogos desktop. Uma instância tem um experimento ativo; não há banco de dados, processamento remoto ou serviço externo de análise.

> Separação útil para manutenção: analysis.py concentra operações numéricas; model.py define quando elas são chamadas e quais estados são válidos; app.js traduz gestos e apresenta resultados.

# 2. Estruturas de dados

new_project(), new_gel() e new_lane() criam dicionários serializáveis em JSON. IDs são os 12 primeiros caracteres hexadecimais de uuid4; a validação detecta duplicatas de gel/amostra. Os limites são 64 géis e 192 amostras por gel.

| Nível | Campos essenciais |
|---|---|
| Projeto | format="panama-blot", version=1, name, gels, reference_gel, sections, audit, settings; pca_ready é adicionado conforme a análise. |
| Gel | id, name, images, lanes, reference_lane, aligned, cross_pairs, notes; lane_review_required identifica geometria inferida pendente. |
| Imagem | name, data em Base64, mode, white, rotation, flip. Durante gravação, data vira referência blob. |
| Amostra | id, name, group, role, include; vetores w e p; w_roi/p_roi; w_trim/p_trim; anchors, anchor_flags, artifacts e baseline. |
| Seção | id, name, start, end, enabled. A lista pertence ao projeto e é compartilhada entre os géis. |
| Resultado por amostra | w/p alinhados quando aplicável; baseline, corrected; listas area, mean, peak_value e warnings. |

## Dados originais, de trabalho e derivados

Os bytes da imagem de origem são preservados. w e p guardam os perfis importados ou a extração atual da ROI; editar uma ROI recalcula esses vetores a partir da imagem preservada. prepared() produz a cópia reamostrada e recortada; máscaras explícitas afetam essa cópia W. analyze_project() deriva os perfis alinhados, fundo e medidas.

Engine contém project, analysis e pca, além de undo_stack, redo_stack, revision, saved_revision, file_path, file_name e cache de prévias. dirty é a desigualdade entre revision e saved_revision: representa revisões, não comparação byte a byte com o último arquivo.

## Configuração inicial

smooth=2; prominence=5%; distance=28; max_peaks=20; search=45; enhance=False. FFT: power=4, center=130, sigma=45, inflect=50, slope=4. PCA: metric="area", normalization="none", scaling="center", log=False.

baseline.mode="none" produz zero numericamente na visualização, mas significa **escolha ainda não feita**. Para PCA, uma linha de base explicitamente zero precisa ser constant com value=0.

# 3. Importação e intensidade

## parse_profiles(text, x_column=False)

Normaliza quebras de linha, descarta linhas vazias e comentários iniciados por #. Escolhe o separador pela primeira linha, com prioridade tabulação, ponto e vírgula e vírgula; sem esses caracteres, divide por espaços. Se a primeira linha não for totalmente numérica, ela é tratada como cabeçalho.

Exige duas ou mais linhas numéricas, até 384 colunas na leitura e comprimentos iguais. A conversão usa float: não interpreta vírgula decimal. Rejeita NaN e infinito. O limite de amostras do projeto é menor: 192. Se x_column=True, exige eixo estritamente crescente e interpola cada coluna para uma grade uniforme com a mesma quantidade de linhas antes de retornar os perfis.

## decode_image(source) e image_info(source)

Decodifica Base64 e abre com Pillow. Rejeita mais de 80 MB, 45 milhões de pixels e imagens com várias páginas. Corrige orientação EXIF, aplica rotação horária solicitada e inversão vertical. Composição alfa usa fundo branco. Para RGB, calcula 0,2126R + 0,7152G + 0,0722B.

O nível de branco padrão é 65535 para modos identificados como 16 bits, ou modo I com máximo acima de 255; nos demais casos, 255. O valor manual precisa ser positivo. A matriz de cálculo permanece em ponto flutuante, normalizada e limitada ao intervalo [0,1]. A prévia é PNG de 8 bits com dimensões de até 1600 × 1600.

```
g = clip(intensidade / white, 0, 1)
Dark bands:       s = 1 - g
Bright bands:     s = g
Optical density:  s = -log10(max(g, 1/65536))
```

## Consequências práticas

O modo de sinal não é um simples ajuste visual: altera a extração. A densidade óptica é aplicada aos valores amostrados antes da média transversal. Em geral, média de -log10(g) é diferente de -log10(média de g). Não há ajuste automático de contraste por amostra.

> O white level pode limitar valores acima dele; a escolha precisa corresponder à aquisição. Preservar 16 bits no cálculo evita a redução da prévia, mas não corrige saturação, faixa efetiva do sensor ou resposta não linear do equipamento.

# 4. Geometria e reamostragem

## extract_from_array(gray, roi, mode)

A ROI contém x1, x2, y1, y2 e width. Para cada posição vertical, interpola o centro entre x1 e x2, permitindo inclinação. Amostra deslocamentos horizontais entre -(width-1)/2 e +(width-1)/2 com map_coordinates(order=1), transforma o sinal e calcula a média de cada linha. A largura é horizontal, não perpendicular à faixa inclinada.

Exige números finitos, y1 < y2, largura válida e todos os extremos dentro da imagem. Usa max(2, round(y2-y1)+1) posições verticais e aproximadamente round(width) amostras transversais.

## detect_lanes() e fit_lane_grid()

A detecção primária usa o percentil 80 do sinal ao longo da altura para formar uma projeção por coluna, suavizada com sigma=max(1, largura/600). find_peaks usa proeminência de 9% da amplitude da projeção. Com quantidade solicitada, retém os picos mais proeminentes e os ordena.

Se faltarem picos, fit_lane_grid() procura uma grade regular usando contraste de bandas na região de 15% a 90% da altura, remoção de fundo lento e percentil 95. Aceita apenas evidência em pelo menos max(3, ceil(N/4)) posições. Sem suporte, propõe espaçamento uniforme de 8% a 92% da largura. Os dois casos inferidos exigem revisão explícita.

A largura sugerida é limitada a 45% da separação mediana, a 1/8 da imagem e aos limites laterais. _pair_protein() coloca ROIs médias entre vizinhas W, ajusta dimensões entre imagens e reduz a largura média para 60%; a última ROI P é compartilhada.

## resample(), prepared() e membrane_edges()

```
origem = linspace(start*(M-1), end*(M-1), 1200)
perfil = interp(origem, arange(M), valores)
eixo de saída = 0, 1, ..., 1199
```

Os recortes são frações entre 0 e 1. A rotina de bordas procura queda da derivada perto do início e subida perto do fim do sinal de borda preta, exige intensidade suficiente e avança cinco pontos para dentro. prepared() aplica o recorte/reinterpolação e, para W, as substituições de artefatos.

> Recortar altera a correspondência entre pixels e eixo 0-1199. Cada geometria nova exige revisar as etapas dependentes. O programa não realiza calibração do eixo de migração em massa molecular.

# 5. Detecção de marcos e FFT

## Suavização e escolha de picos

smooth() usa gaussian_filter1d com mode="nearest"; sigma=0 retorna uma cópia. peak_candidates() suaviza o perfil ou usa landmark_curve() para proteína. A proeminência é prominence/100 vezes a amplitude da curva, e distance impõe a separação mínima.

suggest_anchors() limita a quantidade a max_peaks, retendo os picos mais proeminentes, ordena-os e exclui posições muito próximas aos extremos. A proposta inicial procura máximos. Marcos manuais também podem indicar mínimos, usados depois no pareamento.

## landmark_curve(): realce opcional

Com enhance=True, a curva suavizada é interpolada de 1200 para 1024 pontos, passa por rFFT, recebe um filtro de 513 coeficientes e retorna por irFFT. Depois é interpolada para o eixo original de 1200 posições.

```
x = 0..512
G[x] = exp(-(x-inflect)^2 / (150 * 4^(slope-2)))
C[x] = integral trapezoidal acumulada de G até x
F[x] = 1 + (power-1)*C[x] / max(C[150], 1e-15)
se x > center: F[x] *= exp(-((x-center)/sigma)^2)
se x > 150:    F[x] = 0
realce = irFFT(rFFT(curva_1024) * F)
```

| Parâmetro | Padrão | Papel |
|---|---|---|
| power | 4 | Ganho da rampa integrada. |
| center | 130 | Início do decaimento à direita. |
| sigma | 45 | Largura do decaimento gaussiano. |
| inflect | 50 | Posição central da gaussiana integrada. |
| slope | 4 | Controla a largura da subida. |

Esses números descrevem bins da transformada, não posições de massa molecular. O algoritmo segue a construção de MakeFilterFunc/FilterOneFunc, mas a interpolação inclui os dois extremos; não reproduz exatamente a convenção de índices 1200/1024 do código legado.

A curva filtrada é usada para localizar marcos e desenhar sua edição. A medição é obtida dos perfis de trabalho sem esse realce. Ligar FFT pode mudar correspondências propostas; não aplica o filtro diretamente às intensidades quantificadas.

# 6. Correspondência e warp

## match_anchors()

Para cada marco p do perfil anterior, compara o valor em round(p) com vizinhos a três posições para decidir se procura máximo ou mínimo. Obtém os extremos na curva atual e procura o mais próximo dentro de search, limitado pelos pontos médios entre marcos anteriores e pela ordem dos resultados já encontrados.

Quando não encontra um extremo, mantém a posição proposta e marca anchor_flags=True. Cruzamentos são rejeitados. _alignment("match_anchors") percorre as faixas vizinhas para a direita e para a esquerda, partindo da referência; cada faixa pareada vira a referência da seguinte. Isso permite acompanhar deriva gradual, mas também pode propagar uma correspondência ruim.

## warp(y, target, source)

Recebe marcos do eixo destino e posições correspondentes no perfil fonte. Adiciona os extremos fixos 0 e 1199. Os dois conjuntos precisam ter o mesmo tamanho, valores finitos e ordem estritamente crescente. Para cada ponto do destino, calcula a posição fonte por interpolação linear e amostra y nessa posição.

```
a = [0, *target, 1199]
b = [0, *source, 1199]
source_x = interp(AXIS, a, b)
y_alinhado = interp(source_x, AXIS, y)
```

**Exemplo geométrico:** marcos destino [300, 800] e fonte [330, 780]. A saída em x=300 lê o sinal da fonte em 330; a saída em 800 lê 780. Entre os marcos, as posições variam linearmente. Extremos permanecem fixos.

## Dentro e entre géis

analyze_project() aplica o mesmo warp aos perfis W e P da amostra, usando os marcos da faixa referência. Se o gel não for o mestre e tiver cross_pairs, aplica outro warp, agora do eixo do gel para o eixo do experimento. Somente depois calcula a linha de base.

cross_match propõe pares entre perfis P das referências dos dois géis. Correspondências não encontradas geram mensagem e nota de revisão. O estado cross_pairs guarda apenas coordenadas; não carrega o mesmo bloqueio por flags usado no alinhamento interno. Portanto, os pares entre géis precisam de revisão direta.

Não há compensação de amplitude pelo Jacobiano. Esticar ou comprimir o eixo pode alterar áreas integradas, conforme a convenção adotada do Nirvana. O alinhamento é geométrico e não equivale à normalização de intensidade.

# 7. Fundo, seções e medidas

## baseline(y, config)

**none** retorna zero; **constant** retorna um vetor com value. **rolling** limita a janela a 5-1200, usa percentile_filter(percentile=10, mode="nearest") e suaviza com sigma=max(1, window/10). **manual** ordena pontos, exige pelo menos dois x distintos e interpola linearmente; fora dos pontos extremos, mantém o valor terminal.

O sinal corrigido é W alinhado menos baseline. Os valores negativos são mantidos nesse vetor. O aviso de pontos negativos conta valores abaixo de -1e-10; a integração limita qualquer valor menor que zero a zero.

## Substituições de artefatos

prepared() aplica cada intervalo [a,b] à cópia W já reamostrada: obtém os valores nos extremos e substitui os pontos internos por uma reta. Isso acontece **antes dos warps**. Os intervalos são processados na ordem armazenada; a fonte importada não é regravada por essa operação.

## Seções automáticas

_apply("auto_sections") tira a média das curvas corrected disponíveis no gel escolhido, incluindo amostras fora da PCA. Detecta picos e coloca fronteiras nos mínimos entre pares consecutivos, mais os extremos 0 e 1199. Substitui a lista global de seções; a revisão pode remover intervalos de borda.

## integrate_section(y, start, end)

```
clipped = maximum(y, 0)
xs = [start, pontos inteiros interiores, end]
area = trapezoid(interp(xs, AXIS, clipped), xs)
mean = area / (end - start)
peak_value = 100 * mean
```

A limitação a zero ocorre nos pontos amostrados **antes** da interpolação nos extremos fracionários. Essa escolha pode diferir de integrar uma curva contínua cortada exatamente em seu cruzamento com zero. A largura é end-start, sem somar 1.

validate_project() exige seções dentro de 0-1199 e sem sobreposição, inclusive as desativadas. analyze_project() calcula apenas as seções enabled, na ordem da lista. Áreas, médias e peak values são não normalizados; a normalização é uma etapa posterior da PCA.

> O fator 100 está na macro original, apesar de não aparecer da mesma forma na descrição em prosa. Peak value não significa máximo da curva. A unidade de área usa o eixo reamostrado, sem conversão física.

# 8. Matriz de entrada e PCA

## project_pca(project, analysis)

Seleciona amostras com include=True e vetor W disponível, na ordem dos géis e das faixas. As colunas são as seções enabled. Bloqueia faixas inferidas sem revisão, baseline não definido e perfis pareados cujo gel não esteja alinhado. Para amostras de vários géis, exige cross_pairs nos participantes que não são o mestre.

Escolhe area, mean ou peak_value. Em normalization="reference", localiza o primeiro Standard de cada gel, exige baseline explícita e mapeamentos necessários. Para cada seção j, multiplica a amostra por S_mestre,j / S_gel,j. Os dois padrões precisam ter valores acima de 1e-12. Atribuir Standard exclui a amostra por padrão, mas include pode ser reativado.

## pca_matrix() e ordem das transformações

```
M = matriz de medidas selecionadas
M = normalização por padrão (se escolhida, no chamador)
M = M / soma_de_cada_linha (se total)
M = log1p(M) (se log=True)
mu = média por coluna; sd = desvio amostral (ddof=1)
keep = sd > max(max(abs(M))*1e-12, 1e-14)
Z = (M[:, keep] - mu[keep]) / scales
scales = sd[keep] em autoscale; senão, 1
U, s, Vt = svd(Z, full_matrices=False)
```

Exige matriz bidimensional finita com pelo menos duas linhas e duas colunas e pelo menos duas colunas variáveis. Total rejeita soma de linha não positiva. log1p rejeita negativos. As colunas quase constantes são excluídas e registradas em dropped; kept guarda índices de base zero.

## Resultados numéricos

```
scores = U * s           # cada coluna de U vezes s
loadings = Vt.T
eigenvalues = s**2 / (n-1)
variance = eigenvalues / sum(eigenvalues)
Z = scores @ loadings.T  # reconstrução com todos os PCs
```

O sinal de cada componente é orientado para tornar positivo seu loading de maior magnitude; o mesmo sinal é aplicado aos scores. A SVD reduzida retorna até min(n,p) componentes, podendo incluir componente de variância praticamente zero após centrar.

> matrix no resultado é M após normalização/log, antes de retirar colunas. standardized é Z nas colunas retidas. means tem todas as colunas; scales tem apenas as retidas. Esse detalhe importa para reproduzir a transformação.

# 9. Gráficos e exemplo de PCA

## Exemplo numérico verificável

Considere três amostras com duas seções: [1,2], [2,4], [3,6]. Sem normalização, sem log e com Mean center, as médias são [2,4]. A matriz centrada é [-1,-2], [0,0], [1,2].

```
Covariância = [[1, 2], [2, 4]]
Autovalores = [5, 0]
Loading PC1 = [1/sqrt(5), 2/sqrt(5)]
Scores PC1 = [-sqrt(5), 0, sqrt(5)]
Variância explicada PC1 = 100%
```

Se aplicar Total signal antes de centrar, as três linhas viram [1/3,2/3]. Não há variação; o programa deve rejeitar a PCA. Essa diferença ilustra por que normalização muda a pergunta respondida pelo gráfico.

## Clustering e mapa de calor

O Python calcula linkage(Z, method="average", metric="euclidean", optimal_ordering=True). leaves_list() determina a ordem de amostras. O agrupamento usa todas as colunas de Z, e não só PC1 e PC2. A matriz linkage registra, por fusão, os dois nós unidos, a distância e a quantidade de observações.

O JavaScript extrai as colunas kept de matrix e calcula z scores por coluna com desvio amostral. O heatmap usa a ordem do clustering. A cor é saturada em ±2,5; essa limitação é visual e não muda a PCA. Mesmo com Mean center, o mapa de calor padroniza cada coluna para exibição.

## Scores, loadings e scree

drawPCA() usa SVG. Scores representam linhas da matriz: amostras. Loadings representam colunas retidas: seções. Os seletores pcX e pcY escolhem quais componentes desenhar. O scree mostra frações individuais e soma acumulada; o painel lateral ordena loadings por magnitude absoluta do componente horizontal.

Group determina cores e rótulos, sem participar da SVD ou do clustering. O sinal de um eixo pode mudar entre implementações equivalentes, embora o código atual fixe uma convenção. Componentes com autovalores iguais ou quase iguais ainda podem ter bases diferentes.

> Estas observações descrevem a implementação e suas consequências algébricas. A PCA não realiza teste de hipótese, classificação supervisionada ou estimativa de identidade proteica.

# 10. Edições e invalidação

## Transação em Engine.command()

Para um comando comum, guarda deep copy de project, analysis anterior e mensagem. Executa _apply(), valida o projeto e recalcula a análise. Uma exceção restaura essas referências e é propagada. Só no sucesso registra a cópia em undo_stack, mantém as 40 últimas, limpa redo, incrementa revision e adiciona o evento ao audit.

Edições dos dados removem self.pca e definem pca_ready=False. analysis_options preserva a PCA e invalida apenas os métodos dependentes das opções alteradas. Isso inclui alterações de parâmetros ou metadados. A PCA é executada por comando especial, fora desse fluxo; quando passa a estar pronta, marca pca_ready e incrementa a revisão. Reexecutar a PCA já pronta não cria uma entrada de desfazer.

| Mudança | Consequência relevante |
|---|---|
| Imagem, modo de sinal, ROI, recorte, faixas | invalidate(gel, True) limpa marcos, flags, alinhamento, pares entre géis e linha de base. Rotação/inversão também apagam as ROIs do canal afetado. |
| Marcos ou referência de faixa | Desativa alinhamento e limpa pares do gel. Alterar quantidade na referência apaga correspondências das outras faixas. |
| Geometria/alinhamento do gel mestre | As ações listadas ao final de _apply() limpam cross_pairs dos outros géis. |
| Baseline, seções, máscaras, metadados | Recalcula medidas e invalida a PCA; não necessariamente apaga a geometria. |
| Novo gel mestre | Limpa todos os pares entre géis. |

## Desfazer, refazer e histórico

undo/redo movem snapshots entre pilhas, preservam o audit corrente e acrescentam o evento de reversão. Recalculam analysis e invalidam a PCA; não restauram automaticamente uma PCA antiga. O histórico é cronológico, enquanto o estado do projeto pode voltar atrás.

log() registra hora UTC e parâmetros, excluindo grandes conteúdos data, text, project e files; de files, retém os nomes. public() envia apenas os últimos 200 eventos à interface; o arquivo e a exportação guardam o audit completo. Não é uma trilha assinada ou imutável.

Nem toda mudança força todos os passos: editar marcos mantém a configuração da linha de base, que será recalculada no novo perfil; máscaras também mantêm alinhamento. Revise etapas dependentes mesmo quando o sistema não as bloqueia.

# 11. Projetos e resultados

## pack_project() e unpack_project()

O .panama é um ZIP deflacionado. pack_project() copia o projeto, converte cada imagem Base64 para bytes, calcula SHA-256 e grava um blob por conteúdo distinto em sources/<hash>.bin. No JSON, substitui data por blob. Grava project.json e checksums.json com os hashes do JSON e das fontes.

unpack_project() limita o pacote a 300 MiB e a soma descomprimida a 600 MiB. Confere os hashes declarados, recupera os blobs em Base64 e chama validate_project(). Não extrai os arquivos em caminhos do sistema nem desserializa objetos Python. Hashes detectam corrupção, mas não comprovam autoria ou autenticidade do pacote.

## Salvar e reabrir

save_file() grava em arquivo temporário no mesmo diretório, faz flush/fsync e substitui o destino com os.replace(). Limpa o temporário e só depois atualiza caminho e revisão salva. restore() carrega e calcula em variáveis locais antes de trocar o experimento; falhas preservam o estado atual.

**A PCA não é armazenada como arrays prontos dentro do projeto:** o arquivo guarda estado, parâmetros e pca_ready. Ao abrir, restore() refaz analyze_project() e, se a flag estiver ativa, project_pca(). analyses_ready também determina quais análises de multivariate.py devem ser refeitas. Assim os resultados reaparecem. Versões numéricas futuras podem produzir pequenas diferenças de arredondamento.

| Arquivo no ZIP de exportação | Conteúdo |
|---|---|
| area.csv, mean.csv, peak_value.csv | Medidas não normalizadas de todas as amostras analisáveis, com Group, Role e Include. |
| sections.csv; sample_manifest.csv | Limites/ativação das seções e correspondência de IDs com nomes, géis e avisos. |
| profiles/<gel>_<lane>.csv | Position, Raw_resampled, Aligned, Baseline, Corrected do canal W. Raw_resampled inclui recorte e máscaras. |
| pca_*.csv; pca.json | Quando PCA está atual: scores, loadings, variância, entrada transformada, metadados, centragens, escalas e linkage. |
| audit.json; settings.json; README.txt | Histórico, parâmetros e convenções da exportação. |

CSV usa UTF-8 com BOM. Células de texto iniciadas por =, +, - ou @ recebem apóstrofo para evitar interpretação como fórmula em planilhas. Valores numéricos permanecem numéricos.

# 12. HTTP e integração desktop

## Servidor e sessão

Server liga somente em 127.0.0.1; port=0 solicita porta livre ao sistema. Cria token com secrets.token_urlsafe(32). A URL inicial inclui o token. authorized() exige Host igual a 127.0.0.1:porta e compara o token via compare_digest, recebido no cabeçalho X-Panama-Token ou na query.

| Rota | Resposta / efeito |
|---|---|
| GET /, /index.html, /app.js, /style.css, /icon.svg | Recursos estáticos em lista explícita; não exigem token. |
| GET /api/state | Estado público do experimento; exige sessão. |
| GET /api/project | ZIP .panama e atualização de saved_revision após enviar a resposta. |
| GET /api/export | ZIP de resultados atual. |
| POST /api/command | JSON com action e params; executa Engine.command(). |

As respostas usam no-store, nosniff, no-referrer e Content-Security-Policy com scripts e conexões restritos à origem; imagens data/blob são permitidas. POST exige corpo maior que zero e menor que 400 MiB. Erros de operação retornam JSON com error, normalmente HTTP 400; autenticação inválida retorna 403.

## Desktop.save() e load_experiment()

pywebview expõe a ponte window.pywebview.api. save(kind, content, save_as) admite project, export, plot e sections. Para project com caminho atual e save_as=False, chama save_file() sem novo diálogo. Outros formatos gravam temporário .writing e substituem o destino. load_experiment() usa diálogo .panama e Engine.load_file(). Cancelar retorna cancelled=True.

## Diferenças no navegador

Sem a ponte nativa, app.js obtém project/export pelas rotas e dispara um download. Plot e template usam Blob local. O navegador não sobrescreve automaticamente o arquivo original por um caminho arbitrário. O servidor consegue confirmar o envio, mas não a gravação final do download no disco.

> Essas proteções descrevem um aplicativo local, não uma plataforma multiusuário. Não há autorização por usuário, assinatura de documentos ou mecanismo de auditoria externa. O limite de corpo HTTP se soma aos limites de arquivo, imagem e estrutura.

# 13. Interface e comandos

app.js não usa framework de interface. Funções renderImage(), renderAlign(), renderBaseline(), renderSections(), renderResults() e renderPCA() remontam os painéis e associam eventos. render() atualiza lista de amostras, estado dos botões, aba e mensagens. esc() protege textos inseridos no HTML/SVG.

| Grupo de ações da Engine | Chamadores / finalidade |
|---|---|
| new, demo, open; add_gel, remove_gel; settings, gel | Criação, importação de projeto, organização de géis, parâmetros e referência. |
| image, image_settings, profiles | ImportImage/Profiles e controles de intensidade/orientação. |
| detect_lanes, pair_protein, review_lanes, clear_lanes, roi, lane, remove_lane, auto_crop | Detecção e revisão, geometria, metadados, recortes e máscaras. |
| auto_anchors, match_anchors, anchors, review_anchors, remove_anchor_all, align | Propostas, edição e aplicação de alinhamento interno. |
| cross_match; gel com cross_pairs | Proposta e edição de correspondências entre géis. |
| baseline, auto_sections, sections, auto_pipeline | Fundo, intervalos globais e Auto setup. |
| pca, undo, redo | Execução estatística e reversões especiais. |

## Gestos e coordenadas

svgPoint() transforma coordenadas do ponteiro para o sistema SVG; dataPoint() converte para o domínio do gráfico, limitado a 0-1199. bindPlotEditing() trata seleção, arraste de marcos, pontos e limites das seções. Add snapped busca extremos em até 20 posições ao redor do clique. Os valores exatos também podem ser digitados.

Draw lane calcula uma ROI pelo retângulo desenhado. Em W, omite o ID para criar nova amostra; em P, usa a amostra selecionada. Por isso, editar uma faixa W existente deve usar os campos de Selected lane. O zoom afeta a visualização, sem alterar perfis.

## Arquivos e atualização visual

chooseFile()/fileBase64() fazem a leitura no cliente. download() escolhe ponte desktop ou download do navegador. keepChanges() implementa salvar, descartar ou cancelar antes de New/Load/Demo. exportPlot() serializa o SVG exibido. Templates usam format="panama-sections" e axis=1200.

A função public() retira bytes originais e vetores W/P do JSON público, inclui prévias e perfis para edição e devolve analysis. Não use o estado visual como substituto do projeto completo para persistência.

# 14. Executar e verificar

## Ambiente de desenvolvimento previsto

README.md especifica Python 3.12 ou superior. requirements.txt declara NumPy >=2,<3; SciPy >=1.16,<2; Pillow >=11,<13; pywebview >=6,<7; scikit-learn >=1.5,<2; XlsxWriter >=3.2,<4. pytest e PyInstaller são ferramentas adicionais. Matplotlib é usado pelo ensaio que produz figuras, não pelo aplicativo distribuído.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m pip install pytest pyinstaller
.venv\Scripts\python main.py
.venv\Scripts\python -m pytest -q
powershell -ExecutionPolicy Bypass -File build.ps1
```

main.py aceita --browser, --serve, --port, --experiment e --view (image, align, baseline, sections, results ou pca), além do caminho do projeto como argumento posicional. --smoke-output executa PCA e grava um resumo de verificação sem abrir a interface. --serve mantém o servidor local; --browser abre o navegador.

## Empacotamento

build.ps1 executa PyInstaller em modo windowed e onedir, inclui panama/web, aplica o ícone e exclui bibliotecas não utilizadas. Copia documentação, manuais e licenças para dist/windows-0.2/Panama Blot Analyzer e comprime essa pasta. A pasta versionada preserva a instalação anterior que ainda estiver aberta.

| Conjunto | Evidência coberta |
|---|---|
| test_analysis.py | Reamostragem, integração fracionária e clipping, warp, marcos, extração 16 bits/inclinada, OD, FFT, PCA por covariância/reconstrução. |
| test_workflow.py | Undo/redo, transações rejeitadas, persistência e integridade, exportação, automação, normalização entre géis e autenticação HTTP. |
| test_desktop_files.py | Salvar no mesmo arquivo, Salvar como, carregar e cancelamento de diálogos simulados. |

A versão 0.2.0 passou em **73 verificações automatizadas**, incluindo análise, persistência, Excel, pareamento e HTTP local. O executável Windows também abriu os projetos Mm01a, sintético com todas as análises e sintético pareado, exportando os resultados sem erro. Consulte VALIDATION.txt para detalhes.

Não existe lockfile de dependências nesta base. Para reproduzir numericamente no futuro, registre versões efetivas das bibliotecas e preserve os dados/fontes do ensaio.

# 15. Original, adaptações e limites

| Nirvana / etapa original | Correspondência no Panama |
|---|---|
| NIH PlotProfile; TranslateDataNoRevFunc | Extração de faixa e reamostragem para 1200 posições, com originais preservados. |
| SuperImposeGelsFunc; SmoothAllLanes | Recortes independentes W/P e suavização gaussiana; a suavização difere do Smooth do Igor. |
| MakeFilterFunc / FilterOneFunc | Realce FFT opcional reimplementado; convenção de interpolação difere no eixo. |
| FindNextPeaksFunc | Busca em faixas adjacentes, com sinalização de correspondências não encontradas. |
| CalculateScaleFunc / RescaleOneFunc | Warp linear por trechos, aplicado aos canais W/P sem compensação de amplitude. |
| SubtractBaselineFunc; EliminatePeakArtefact | Fundo constante/manual e substituição linear de artefato; rolling é uma opção nova. |
| CalculateAreasFunc / CalcAreasFunc | Integração com negativos limitados a zero; peak_value=100×area/largura. |
| CorrFactorsA / CorrFactorsPV | Correção por seção pela razão entre padrões dos géis. |
| Nirvana Stat / Mathematica | PCA nativa por SVD, com heatmap e clustering; não é execução do código Mathematica antigo. |

O original considerava 28 faixas W e 27 P pela cassete experimental. O Panama generaliza esse limite mantendo o compartilhamento da última proteína. A diferença encontrada entre as macros 2.1 e 2.2 fornecidas envolve cópias de vetores de alinhamento entre géis com sufixos incrementais; aqui, estado e histórico ficam no projeto.

## O que foi e o que não foi demonstrado

O Mm01a revisado contém 18 amostras, seis marcos e 14 seções entre 250 e 1050. O registro mostra PC1=76,7%, PC2=13,3%, melhora da correlação proteica e restauração de medidas após desfazer. Não havia saída histórica correspondente para comparação numérica independente.

O pacote antigo contém a interface estatística de troca com Mathematica, mas não a implementação de PCA correspondente. Não estão implementados leitores de binários Igor/.bwav, ELISA ou calibração de massa molecular/aquisição. A distribuição foi construída e verificada em Windows; macOS/Linux não foram validados neste projeto.

# 16. Referências e manutenção

## Fontes locais efetivamente utilizadas

**Código:** main.py; panama/analysis.py, model.py, server.py; panama/web/app.js e index.html. **Execução:** requirements.txt, build.ps1 e Panama Blot Analyzer.spec. **Verificação:** tests/test_analysis.py, test_workflow.py e test_desktop_files.py.

**Documentação atual:** README.md, METHODS.md, VALIDATION.txt, experiments/Mm01a/READ-ME.txt, trial-results.json e tools/write_mm01a_manual.py. Os números do Mm01a são do ensaio registrado, não de uma nova análise biológica.

**Material histórico:** Nirvana 2.1 Help e General Introduction (incluindo versões Word), Nirvana 2.1/2.2 Macro, NIH Quantifying, Load_from_NIHImage, Utilities e Nirvana Stat, preservados em reference/original e reference/text. A relação de entradas do arquivo fornecido está em reference/archive-manifest.json. Autoria histórica: Matthias Haury e Alberto Nobrega.

## Identificação desta base por SHA-256

main.py

```
aba85b27bcb6d95b65b2544c0e0bf686
22e91134b1d3cba422f867a9d6a66c8d
```

panama/analysis.py

```
8f779f985a8b376d40f7c1cb7374c8ff
5c3c239878c845a8a12b94aa7b4fed7c
```

panama/model.py

```
478f34f6841d16aef715da927093fe46
04c5bce4057bf2e1602fe8b507716940
```

panama/server.py

```
eaf6648785168d0ebd2c347814b17181
5d76b9e7cd2d4e740a3f27dc97c070a5
```

panama/web/app.js

```
811f58f1b935eb7ce09609800c219e27
d5adeb1031d8f04509f367540b743383
```

Cada hash está quebrado em duas linhas de 32 caracteres; concatene as linhas para obter os 64 caracteres. Não identifica dependências nem substitui um commit de repositório.

## Como evoluir sem perder rastreabilidade

Para alterar um algoritmo, registre a convenção numérica em METHODS.md, mantenha uma função verificável em analysis.py e exercite casos analíticos conhecidos. Para novas edições, atualize _apply(), validação, invalidação e interface em conjunto. Se o formato persistido mudar, implemente migração/versionamento e teste projetos antigos.

O gerador destes dois documentos é tools/write_portuguese_manuals.py. Ao atualizar a aplicação, confira os rótulos e os detalhes documentados antes de regenerar os PDFs. As referências de função são mais estáveis que números de linha.

# 17. Novas análises e Excel

## Matriz compartilhada

project_matrix() em analysis.py valida a preparação, seleciona amostras incluídas e seções ativas, calcula a métrica escolhida e aplica normalização/log. Retorna raw, matrix, metadados, IDs e settings. project_pca() reaproveita essa entrada; PCA matrix exporta exatamente standardized.

## multivariate.py

Heatmap mantém constantes, calcula z scores com ddof=1 ou usa valores de entrada; ordenação opcional por distância euclidiana e average linkage nos valores exibidos. KMeans usa 20 inicializações, k-means++, seed explícito e todas as seções variáveis após centrar/escalar. O silhouette e a curva de inércia auxiliam a inspeção; o gráfico é uma projeção PCA.

LDA usa solver SVD e pipeline VarianceThreshold + StandardScaler + LinearDiscriminantAnalysis. StratifiedKFold com shuffle e seed usa min(5, menor grupo) folds. Cada transformação aprendida é ajustada dentro do treino. cross_val_predict fornece probabilidades e matriz de confusão fora da amostra de treino; outro ajuste completo gera os eixos exibidos. Pair IDs repetidos são rejeitados para evitar tratar medidas dependentes como amostras independentes.

## workbook.py e ponte desktop

XlsxWriter gera .xlsx em memória, sem interpretar rótulos como fórmulas ou URLs. Matrizes têm IDs de amostra e gel, nome e grupo antes das colunas numéricas. Samples inclui Pair e exclusões; Sections registra limites. Resultados prontos ganham planilhas próprias. Desktop.save("excel") sempre abre diálogo de destino e grava por arquivo temporário seguido de substituição atômica.

## Estado e persistência

Engine.multivariate guarda os resultados atuais. analyses_ready e analysis_options ficam no .panama; restore() recalcula os métodos prontos antes de substituir o estado. Edições dos dados invalidam resultados; alterações de opções invalidam apenas os métodos dependentes. Undo/redo limpa análises derivadas. Projetos da versão 0.1 continuam abrindo, com Pair vazio por padrão.

Novos arquivos: panama/multivariate.py, panama/workbook.py, panama/web/analysis.js e tests/test_multivariate.py. Dependências de execução adicionadas: scikit-learn e XlsxWriter; SciPy fornece a inferência estatística.

# 18. Inferência e verificação

## Convenções estatísticas

A inferência usa matrix antes de centrar/escalar, após normalização/log. Resumos usam SD amostral e intervalo t para a média. Welch t usa equal_var=False; Welch ANOVA usa f_oneway(equal_var=False). Mann-Whitney e Kruskal-Wallis são testes por postos. Comparações requerem rótulos completos e no mínimo duas observações por grupo selecionado.

## Pareamento explícito

paired_t e wilcoxon escolhem Group A/B e constroem índices por Pair ID. Conjuntos de IDs devem ser idênticos, com uma ocorrência por grupo e no mínimo dois pares. Os arrays são ordenados pelos IDs antes do cálculo. ttest_rel fornece diferença média A-B e IC de 95%; wilcoxon usa diferenças A-B, alternative="two-sided", method="auto" e zero_method="wilcox". A diferença média exibida para Wilcoxon é descritiva, sem IC de média.

Diferença pareada com variância zero fica indefinida no teste t; todas as diferenças zero ficam indefinidas em Wilcoxon. Welch t admite um grupo constante quando o outro varia; ANOVA requer variância positiva em cada grupo. Valores não finitos não são convertidos em p=0. O método de Benjamini-Hochberg ajusta os p válidos das seções nesta execução.

## Testes e limites

tests/test_multivariate.py verifica separação e determinismo de K-means, pré-processamento da LDA dentro dos folds, constantes no heatmap, p/q/IC contra SciPy, pares fora da ordem, rejeição de IDs inválidos, exportação da matriz exata e restauração de análises. Testes desktop conferem que Excel sempre solicita destino. O ensaio real Mm01a mantém 18 × 14 e PC1=76,7%, PC2=13,3%; grupos e pares permanecem vazios.

As análises são exploratórias e dependem do desenho experimental. Validação cruzada interna não equivale a validação externa; múltiplas execuções exploratórias exigem um plano mais amplo de multiplicidade. Não há modelos de mais de duas condições repetidas, nem prova de equivalência numérica com uma saída histórica do Nirvana.

## Referências primárias

SciPy: stats.ttest_ind, f_oneway, mannwhitneyu, kruskal, ttest_rel, wilcoxon e false_discovery_control. Scikit-learn: KMeans, LinearDiscriminantAnalysis, StratifiedKFold e Pipeline. Links oficiais e parâmetros estão em METHODS.md.